

FIRST TERM EXAMINATION [FEB. 2017] SIXTH SEMESTER [B.TECH] COMPILER DESIGN [ETCS-302]

Time : 1.5 Hrs.

M.M. : 30

Note: Attempt three questions. Question No. 1. is compulsory. Each question carries 10 marks.

Q.1. (a) Mention the role of Lexical Analyzer in compiler Design. (2)

Ans. The main task is to read the input characters and produce as output sequence of tokens that the parser uses for syntax analysis.

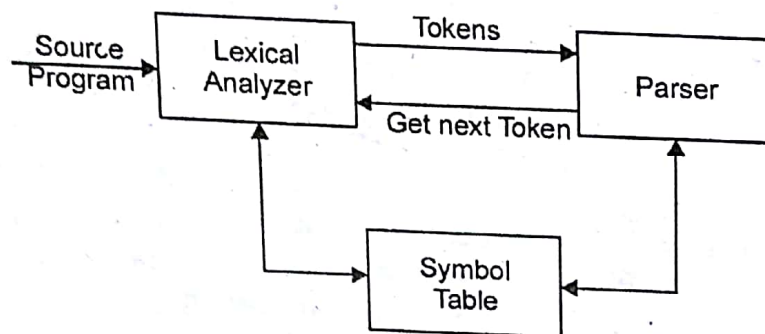


Fig. Role of the lexical analyzer diagram

Up on receiving a "get next token" command from the parser, the lexical analyzer reads input characters until it can identify the next token.

Its secondary tasks are:

- One task is stripping out from the source program comments and white space in the form of blank, tab, new line characters.
- Another task is correlating error messages from the compiler with the source program.

Sometimes lexical analyzer is divided in to cascade of two phases.

- (1) Scanning
- (2) Lexical analysis.

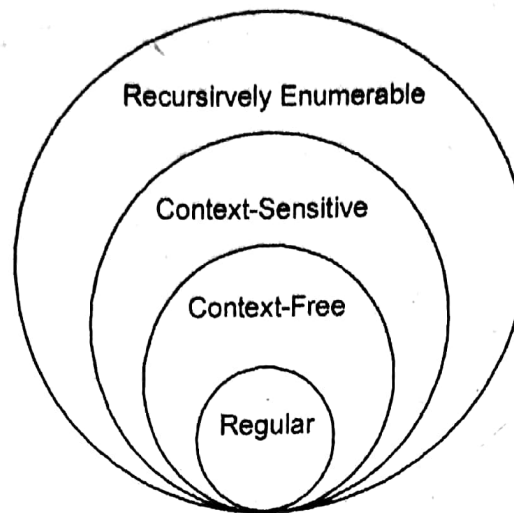
The scanner is responsible for doing simple tasks, while the lexical analyzer proper does the more complex operations.

Q.1. (b) Explain Chomsky classification of languages. (2)

Ans. According to Noam Chomsky, there are four types of grammars "Type 0, Type 1, Type 2, and Type 3. The following table shows how they differ from each other –

Grammar Type	Grammar Accepted	Language Accepted	Automaton
1. Type 0	Unrestricted grammar	Recursively enumerable language	Turing Machine
2. Type 1	Context-sensitive grammar	Context-sensitive language	Linear-bounded automaton
3. Type 2	Context-free grammar	Context-free language	Pushdown automaton
4. Type 3	Regular grammar	Regular language	Finite state automaton

Take a look at the following illustration. It shows the scope of each type of grammar



Type - 3 Grammar

Type-3 grammars generate regular languages. Type-3 grammars must have a single non-terminal on the left-hand side and a right-hand side consisting of a single terminal or single terminal followed by a single non-terminal.

The productions must be in the form $X \rightarrow a$ or $X \rightarrow aY$

where $X, Y \in N$ (Non terminal)

and $a \in T$ (Terminal)

The rule $S \rightarrow \epsilon$ is allowed if S does not appear on the right side of any rule.

Example

$X \rightarrow \epsilon$

$X \rightarrow a \mid aY$

$Y \rightarrow b$

Type - 2 Grammar

Type-2 grammars generate context-free languages.

The productions must be in the form $A \rightarrow \gamma$

where $A \in N$ (Non terminal)

and $\gamma \in (T \cup N)^*$ (String of terminals and non-terminals).

These languages generated by these grammars are recognized by a non-deterministic pushdown automaton.

Example

$S \rightarrow Xa$

$X \rightarrow a$

$X \rightarrow aX$

$X \rightarrow abc$

$X \rightarrow \epsilon$

Type - 1 Grammar

Type-1 grammars generate context-sensitive languages. The productions must be in the form

$\alpha A \beta \rightarrow \alpha \gamma \beta$

where $A \in N$ (Non-terminal)

and $\alpha, \beta, \gamma \in (T \cup N)^*$ (Strings of terminals and non-terminals)

The strings α and β may be empty, but γ must be non-empty.

The rule $S \rightarrow \epsilon$ is allowed if S does not appear on the right side of any rule. The languages generated by these grammars are recognized by a linear bounded automaton.

Example

$AB \rightarrow AbBc$

$A \rightarrow bcA$

$B \rightarrow b$

Type - 0 Grammar

Type-0 grammars generate recursively enumerable languages. The productions have no restrictions. They are any phase structure grammar including all formal grammars.

They generate the languages that are recognized by a Turing machine.

The productions can be in the form of $\alpha \rightarrow \beta$ where α is a string of terminals and nonterminals with at least one non-terminal and α cannot be null. β is a string of terminals and non-terminals.

Example

$S \rightarrow ACaB$

$Bc \rightarrow acB$

$CB \rightarrow DB$

$aD \rightarrow Db$

Q.1. (c) What are the three methods to construct LR Parsing tables. Which method is the most powerful? Justify. (2)

Ans. There are three widely used algorithms available for constructing an LR parser:

- SLR(1) - Simple LR
 - o Works on smallest class of grammar.
 - o Few number of states, hence very small table.
 - o Simple and fast construction.
- LR(1) - LR parser
 - o Also called as Canonical LR parser.
 - o Works on complete set of LR(1) Grammar.
 - o Generates large table and large number of states.
 - o Slow construction.
- LALR(1) - Look ahead LR parser
 - o Works on intermediate size of grammar.
 - o Number of states are same as in SLR(1).

Q.1. (d) An ambiguous grammar can never be LR. Justify the statement with an example. (2)

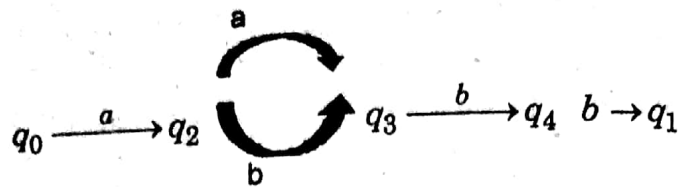
Ans: An ambiguous grammar can never be LR(k) for any k, because LR(k) algorithm aren't designed to handle ambiguous grammars. It would get stuck into undecidability problem, if employed upon an ambiguous grammar, no matter how large the constant k is.

Q.1. (e) Design the NFA for the regular Expression $a(a+b)^*bb$ (2)

Ans.

$$q_0 \xrightarrow{a(a+b)^*bb} q_1$$

$$q_0 \xrightarrow{a} q_2 \xrightarrow{(a+b)^*} q_3 \xrightarrow{b} q_4 \xrightarrow{b} q_1$$



Q.2. (a) Let G be CFG:

$$S \rightarrow bB \mid aA$$

$$A \rightarrow b \mid bS \mid aAA$$

$$B \rightarrow a \mid aS \mid bBB$$

(5)

For the string bbaababa find

(i) Left Most Derivative

(ii) RightMost Derivative

(iii) Parse Tree

Ans. (i) Left Most Derivative: $S \rightarrow bB$

$bbBB$

$bbaSB$

$bbaaB$

$bbaabBB$

$bbaabaSB$

$bbaababa$

(ii) RightMost Derivative:

$S \rightarrow bB$

$bbBB$

$bbBa$

$bbaSa$

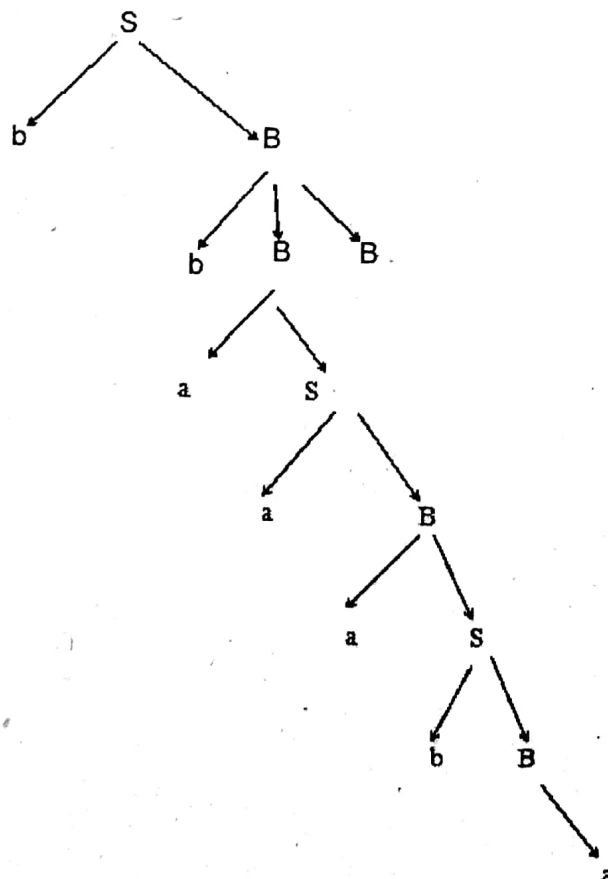
$bbaaAa$

$bbaabSa$

$bbaabaAa$

$bbaababa$

(iii) Parse tree



Q.2. (b) Construct predictive parsing table for the following grammar (S is start Symbol)

$$S \rightarrow i C t S S' | a$$

$$S' \rightarrow e S | \epsilon$$

$$C \rightarrow b$$

(5)

Ans. Step 1. Eliminate left-Recursion and Left Factor the grammar if required. As, there is no left. Recursion and grammar is already left Factored.

Step 2. Computation of FIRST

(5)

$$(i) \quad S \rightarrow i C t S S' | a$$

$$\therefore \text{FIRST}(S) = \{i, a\}$$

$$(ii) \quad S' \rightarrow e S | \epsilon$$

$$\therefore \text{FIRST}(S') = \{e, \epsilon\}$$

$$(iii) \quad C \rightarrow b$$

$$\therefore \text{FIRST}(C) = \{b\}$$

$$\begin{array}{l} \text{FIRST}(S) = \{i, a\} \\ \text{FIRST}(S') = \{e, \epsilon\} \\ \text{FIRST}(C) = \{b\} \end{array}$$

Step 3. Computation of FOLLOW:
Applying Rule (1) of FOLLOW

$$\text{FOLLOW}(S) = \{\$ \}$$

(1)

$$S \rightarrow i C t S S'$$

Applying Rule (2) of FOLLOW.

$S \rightarrow$	i	C	$t S S'$
$A \rightarrow$	α	B	β

$$\therefore \text{FIRST}(\beta) = \text{FIRST}(t S S') = \{t\}$$

$$\therefore \text{FIRST}(\beta) \text{ does not contain } \epsilon.$$

$$\therefore \text{By Rule (2a) FOLLOW}(C) = \{\text{FIRST}(t S S')\}$$

$$\therefore \text{FOLLOW}(C) = \{t\}$$

Again, Apply Rule (2) on different combination

$S \rightarrow$	$i C t$	S	S'
$A \rightarrow$	α	B	β

$$\text{FIRST}(\beta) = \text{FIRST}(S') = \{e, \epsilon\} \text{ contain } \epsilon.$$

$$\text{By Rule (2b) FOLLOWS}(S) = \text{FIRST}(S') - \{\epsilon\} \cup \text{FOLLOW}(S)$$

$$\text{FOLLOW}(S) = \{e, \epsilon\} - \{\epsilon\} \cup \text{FOLLOWS}(S)$$

$$\text{FOLLOW}(S) = \{e\} \cup \text{FOLLOW}(S)$$

Applying Rule (3) of Follow

$S \rightarrow$	$i C t S$	S'
$A \rightarrow$	α	B

$$A = S, \alpha = i C t S, B = S'$$

$$\therefore \text{FOLLOW}(S') = \{\text{FOLLOW}(S)\}$$

(2)

$$S' \rightarrow eS$$

Rule (2) can not be applied on this Production.

Applying Rule (3) of FOLLOW

$S' \rightarrow$	e	S
$A \rightarrow$	α	B

$$\text{FOLLOW}(S) = \{\text{FOLLOW}(S')\}$$

The Renaming Production $S \rightarrow a$, $S' \rightarrow \epsilon$ and $C \rightarrow b$ do not match with an Rule
Combining Statements (1) to (5)

$$\text{FOLLOWS}(S) = \{\$ \}$$

$$\text{FOLLOWS}(C) = \{t\}$$

$$\text{FOLLOW}(S) = \{e\} \cup \text{FOLLOW}(S)$$

$$\text{FOLLOW}(S') = \{\text{FOLLOW}(S)\}$$

$$\text{FOLLOW}(S) = \{\text{FOLLOW}(S')\}$$

$$\therefore \text{FOLLOW}(S) = \text{FOLLOW}(S) = \{\$, e\}$$

$$\text{FOLLOW}(C) = \{t\}$$

Step 4: Construction of Predictive Parsing Table. Put all Non- terminals S'C'S
Row wise and All terminals. i, a, b, e, \$, t Column wise

(1)

$$S \rightarrow i C t S S'$$

Comparing $S \rightarrow i C t S S'$ with $A \rightarrow \alpha$

$S \rightarrow$	$i C t S S'$
$A \rightarrow$	α

$$\text{FIRST}(\alpha) = \text{FIRST}(i C t S S') = \{i\}$$

Applying Rule (1) of predictive Parsing Table:-

Add $S \rightarrow i C t S S'$ to $M[S, i]$

Write $S \rightarrow i C t S S'$ in front of Row (S) and Column (i)

...(1)

$$(2) S \rightarrow a$$

Comparing it with

$$A \rightarrow \alpha$$

$S \rightarrow$	a
$A \rightarrow$	α

$\therefore$

FIRST

$$A = S, \alpha = a$$

$$(\alpha) = \text{FIRST}(a) = \{a\}$$

$\therefore$ Put $S \rightarrow a$ onto $M[S, a]$

Write $S \rightarrow a$ in front of Row (s), column (a)

$$(3) S' \rightarrow eS$$

Comparing it with $A \rightarrow \alpha$

$S' \rightarrow$	eS
$A \rightarrow$	α

$$A = S', \alpha = eS$$

$$\text{FIRST}(\alpha) = \text{FIRST}(eS) = \{e\}$$

$$\text{Add } S' \rightarrow eS \text{ to } M[S', e]$$

Write $S' \rightarrow eS$ in front of Row (S'), column (e)

(4) $S' \rightarrow \epsilon$

Comparing it with

$$A \rightarrow \alpha$$

$S' \rightarrow$	ϵ
$A \rightarrow$	α

$$A = S', \alpha = \epsilon$$

$$\text{FIRST}(\alpha) = \text{FIRST}(\epsilon) = \{\epsilon\}$$

∴ Applying Rule (2) of Predictive Parsing Table. i.e. If ϵ is in $\text{FIRST}(\alpha)$

Then FOLLOW

$$(S') = \{e, \$\}$$

∴ Add

$$S' \rightarrow \epsilon \text{ to } M[S', e] \text{ } M[S', \$]$$

write $S' \rightarrow \epsilon$ in front of Row (S') Column $\{(e, \$)\}$

(5)

Comparing it with

$$C \rightarrow b$$

$$A \rightarrow \alpha$$

$C \rightarrow$	b
$A \rightarrow$	α

$$\text{First}(\alpha) = \text{FIRST}(b) = \{b\}$$

∴ Add $C \rightarrow b$ to $M[C, b]$

write $C \rightarrow b$ in front of Row (C), Column (b)

∴ Combining statements (1) to (5) we get following Predictive Parsing table.

	a	b	e	i	t	\$
S	$S \rightarrow a$			$S \rightarrow i C t S S'$		
S'			$S' \rightarrow eS$ $S' \rightarrow \epsilon$			$S' \rightarrow \epsilon$
C		$C \rightarrow b$				

This Grammar is not LL (1) because In Row (S') and column (e) we have multiple Entries for productions

$$S' \rightarrow eS$$

$$S' \rightarrow \epsilon$$

$$S' \rightarrow eS \text{ appears as } \text{FIRST}(S') = \{e\}$$

$$S' \rightarrow \epsilon \text{ appears as } \text{FOLLOW}(S') = \{\epsilon\}$$

This Grammar is not LL (1) Grammar.

Q.3. (a) Construct SLR parsing Parsing table for the following grammar: (5)

$$S \rightarrow xAy | xBy | xAz$$

$$A \rightarrow aS | q$$

$$B \rightarrow q$$

Ans: Step 1: Construct augmented Grammar

$$(0) S' \rightarrow S$$

$$(1) S \rightarrow xAy$$

$$(2) S \rightarrow xBy$$

$$(3) S \rightarrow xAz$$

$$(4) A \rightarrow aS$$

$$(5) B \rightarrow q$$

Step 2. Find Closure & goto functions to construct LR (0) items. Here Boxes represent New States and Circles represent the repeating state.

Closure ($S' \rightarrow \bullet S$)

I_0 :
 $S' \rightarrow \bullet S$
 $S \rightarrow \bullet x A y$
 $S \rightarrow \bullet x B y$
 $S \rightarrow \bullet x A z$

$I_4 = \text{goto}(I_0, B)$
 $S \rightarrow x B \bullet y$

$I_8 = \text{goto}(I_4, y)$
 $S \rightarrow x B y \bullet$

$I_1 = \text{goto}(I_0, S)$
 $S' \rightarrow S \bullet$

$I_5 = \text{goto}(I_2, q)$
 $A \rightarrow q \bullet S$
 $A \rightarrow q \bullet$
 $B \rightarrow q \bullet$
 $S \rightarrow \bullet x A y$
 $S \rightarrow \bullet x B y$
 $S \rightarrow \bullet x A z$

$I_9 = \text{goto}(I_5, S)$
 $A \rightarrow q S \bullet$

$I_2 = \text{goto}(I_0, x)$
 $S \rightarrow x \bullet A y$
 $S \rightarrow x \bullet B y$
 $S \rightarrow x \bullet A z$
 $A \rightarrow \bullet q S$
 $A \rightarrow \bullet q$
 $B \rightarrow \bullet q$

$I_6 = \text{goto}(I_5, y)$
 $S \rightarrow x A y \bullet$

$I_2 = \text{goto}(I_5, x)$
 $S \rightarrow x \bullet A y$
 $S \rightarrow x \bullet B y$
 $S \rightarrow \bullet A z$
 $A \rightarrow \bullet q$
 $B \rightarrow \bullet q$

$I_3 = \text{goto}(I_2, A)$
 $S \rightarrow x A \bullet y$
 $S \rightarrow x A \bullet z$

$I_7 = \text{goto}(I_3, z)$
 $S \rightarrow x A z \bullet$

Step 3. Computation of FOLLOW

1. $S \rightarrow x A y$

$\text{Follow}(S) = \{\$ \}$

Applying Rules (2a) of FOLLOW

(Comparing $S \rightarrow x a y$ with $A \rightarrow \alpha B \beta$)

$\therefore \text{FIRST}(\beta) = \text{FIRST}(y) = \{y\}$

$\therefore \text{FOLLOW}(A) = \{y\}$

Rule (3) cannot be applied.

As, $S \rightarrow x A y$ cannot be compared with $A \rightarrow \alpha B$

2. $S \rightarrow x B y$

Applying Rule (2a) of FOLLOW

comparing $S \rightarrow x B y$ with $A \rightarrow \alpha B \beta$

$\therefore \text{FIRST}(\beta) = \{y\}$

Here Boxes represent

$$\therefore \text{FOLLOW}(B) = \{y\}$$

Rule (3) cannot be applied.

$$3. S \rightarrow xAy$$

Applying Rule (2a) of Follow

$$\text{First}(\beta) = \{z\}$$

$$\therefore \text{FOLLOW}(A) = \{z\}$$

Rule (3) cannot be applied.

$$4. A \rightarrow qS$$

Rule (2a) cannot be applied. As $A \rightarrow qS$ cannot be compared with $A \rightarrow \alpha B \beta$

Applying Rule (3)

Comparing $A - qS$ with $A \rightarrow \alpha\beta$

∴

$$A = A, \alpha = q, B = S$$

$$\therefore \text{FOLLOW}(S) = \{\text{FOLLOW}(A)\}$$

Rule (2a) and Rule (3) of FOLLOW cannot be applied on production $A \rightarrow q$ and $B \rightarrow q$. Combining statements 1 to 5

$$\text{FOLLOW}(A) = \{y, z\}$$

$$\text{FOLLOW}(s) = \{\$, y, z\}$$

$$\text{FOLLOW}(B) = \{y\}$$

Step 4: Construction of SLR (1) Parsing Table:

States	Action					goto		
	x	y	z	q	\$	S	A	B
0	s2	r3/r4				1		
1					accept			
2				s5			3	4
3		s6	s7					
4		s8						
5	s2	r5/r6	r5			9		
6		r1	r1		r1			
7		r3	r3		r3			
8		r2	r2		r2			
9		r4	r4					

Q.3. (b) Discuss ambiguity of a CFG. How Ambiguity can be removed from the grammar? State suitable example. (5)

Ans. If a context free grammar G has more than one derivation tree for string $w \in L(G)$, it is called an **ambiguous grammar**. There exist multiple right-most or left-most derivations for some string generated from that grammar.

Problem

To check whether the grammar G with production rules-

$$X \rightarrow X+X \mid X*X \mid X^* \mid a \text{ is ambiguous or not.}$$

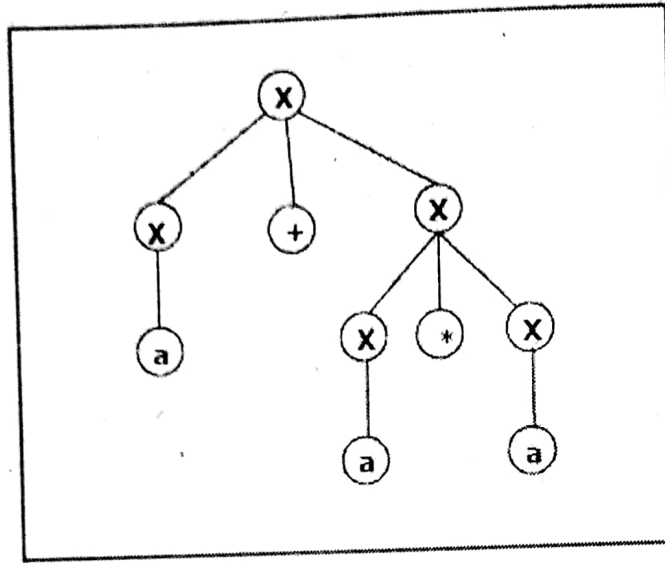
Solution

Let's find out the derivation tree for the string "a+a*a". It has two leftmost derivations.

Derivation 1 -

$X \rightarrow X+X \rightarrow a+X \rightarrow a+X*X \rightarrow a+a*X \rightarrow a+a*a$

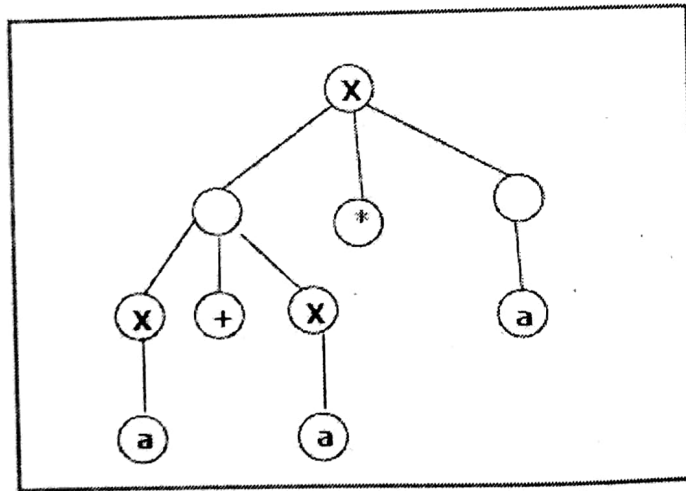
Parse tree 1 -



Derivation 2-

$X \rightarrow X*X \rightarrow X+X*X \rightarrow a+X*X \rightarrow a+a*X \rightarrow a+a*a$

Parse tree 2 -



As there are two parse trees for a single string "a+a*a", the grammar G is ambiguous.

Removal of Ambiguity:

1. Precedence of Operators is not respected.
2. Sequence of identical operators can group either from left or from right. We would see two different parse tree for the above expression. Since addition is associative, it doesn't matter w

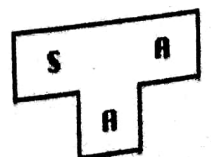
Q.4. Write short notes on following:

(a) Bootstrapping and Cross Compiler

Ans: A compiler is characterized by three languages:

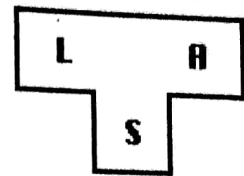
1. Source Language
2. Target Language
3. Implementation Language

(2.5)



s two leftmost

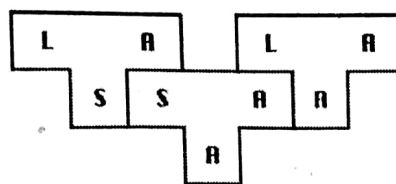
Notation: ${}^S C_I^T$ represents a compiler for Source S, Target T, implemented in I. The T-diagram shown above is also used to depict the same compiler.



To create a new language, L, for machine A:

1. Create ${}^S C_A^A$, a compiler for a subset, S, of the desired language, L, using language A which runs on machine A. (Language A may be assembly language.)
2. Create ${}^L C_S^A$, a compiler for language L written in a subset of L.
3. Compile ${}^L C_S^A$ using ${}^S C_A^A$ to obtain ${}^L C_A^A$, a compiler for language L, which runs on machine A and produces code for machine A.

$${}^L C_S^A \rightarrow {}^S C_A^A \rightarrow {}^L C_A^A$$

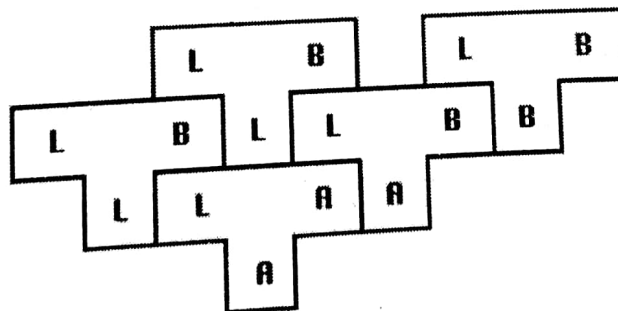


The process illustrated by the T-diagrams is called *bootstrapping* and can be summarized by the equation:

$$L_S \wedge + S_A \wedge = L_A \wedge$$

To produce a compiler for a different machine B:

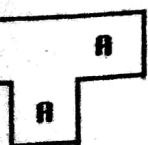
1. Convert ${}^L C_S^A$ into ${}^L C_L^B$ (by hand, if necessary). Recall that language S is a subset of language L.
2. Compile ${}^L C_L^B$ to produce ${}^L C_A^B$, a *cross-compiler* for L which runs on machine A and produces code for machine B.
3. Compile ${}^L C_L^B$ with the cross-compiler to produce ${}^L C_B^B$, a compiler for language L which runs on machine B.



s ambiguous.

ht. We would
ssociative, it

(2.5)



Cross Compiler is a compiler capable of creating executable code for a platform other than the one on which the compiler is running. For example, a compiler that runs on a Windows 7 PC but generates code that runs on Android smartphone is a cross compiler.

A cross compiler is necessary to compile for multiple platforms from one machine. A platform could be infeasible for a compiler to run on, such as for the microcontroller of an embedded system because those systems contain no operating system.

In paravirtualization one machine runs many operating systems, and a cross compiler could generate an executable for each of them from one main source.

Cross compilers are not to be confused with source-to-source compilers. A cross compiler is for cross-platform software development of binary code, while a source-to-source compiler translates from one programming language to another in text code. Both are programming tools.

Uses of cross compilers

The fundamental use of a cross compiler is to separate the build environment from target environment. This is useful in a number of situations:

- Embedded computers where a device has extremely limited resources. For example, a microwave oven will have an extremely small computer to read its touchpad and door sensor, provide output to a digital display and speaker, and to control the machinery for cooking food. This computer will not be powerful enough to run a compiler, a file system, or a development environment. Since debugging and testing may also require more resources than are available on an embedded system, cross-compilation can be less involved and less prone to errors than native compilation.
- Compiling for multiple machines. For example, a company may wish to support several different versions of an operating system or to support several different operating systems. By using a cross compiler, a single build environment can be set up to compile for each of these targets.
- Compiling on a server farm. Similar to compiling for multiple machines, a complicated build that involves many compile operations can be executed across any machine that is free, regardless of its underlying hardware or the operating system version that it is running.
- Bootstrapping to a new platform. When developing software for a new platform, or the emulator of a future platform, one uses a cross compiler to compile necessary tools such as the operating system and a native compiler.
- Compiling native code for emulators for older now-obsolete platforms like the Commodore 64 or Apple II by enthusiasts who use cross compilers that run on a current platform (such as Aztec C's MS-DOS 6502 cross compilers running under Windows XP).

Use of virtual machines (such as Java's JVM) resolves some of the reasons for which cross compilers were developed. The virtual machine paradigm allows the same compiler output to be used across multiple target systems, although this is not always ideal because virtual machines are often slower and the compiled program can only be run on computers with that virtual machine.

Q.4. (b) Difference between compiler and interpreter

(2.5)

Ans.

Interpreter	Compiler
1. Translates program one statement at a time.	Scans the entire program and translates it as a whole into machine code.
2. It takes less amount of time to analyze the source code but the overall execution time is slower.	It takes large amount of time to analyze the source code but the overall execution time is comparatively faster.
3. No intermediate object code is generated, hence are memory efficient.	Generates intermediate object code which further requires linking, hence requires more memory.
4. Continues translating the program until the first error is met, in which case it stops. Hence debugging is easy.	It generates the error message only after scanning the whole program. Hence debugging is comparatively hard.
5. Programming language like Python	Programming language like C, C++ use

Q.4. (c) Handle and Handle Pruning**Ans: Handles:**

(2.5)

• A *handle* of a string is a substring that matches the right side of a production, whose reduction to the nonterminal on the left side of the production represents one step along the reverse of a rightmost derivation.

• **Precise definition of a handle:**

• A *handle* of a right-sentential form γ is a production $A \rightarrow \beta$ and a position of γ where the string β may be found and replaced by A to produce the previous right-sentential form in a rightmost derivation of γ .

• i.e., if $S \alpha A w \alpha \beta w$, then $A \rightarrow \beta$ in the position following α is a handle of $\alpha \beta w$.

• The string w to the right of the handle contains only terminal symbols.

• In the example above, $abbcde$ is a right sentential form whose handle is $A \rightarrow b$ at position 2. Likewise, $aAbcde$ is a right sentential form whose handle is $A \rightarrow Abc$ at position 2.

• **Handle Pruning:**

• A rightmost derivation in reverse can be obtained by *handle pruning*.

• i.e., start with a string of terminals w that is to parse. If w is a sentence of the grammar at hand, then $w \Rightarrow \gamma_n$, where $\rightarrow \gamma_n$ is the n th right sentential form of some as yet unknown rightmost derivation.

$$S = \gamma_0 \gamma_1 \gamma_2 \dots \gamma_{n-1} \gamma_n = w.$$

Example for right sentential form and handle for grammar

$E \rightarrow E + E$

$E \rightarrow E * E$

$E \rightarrow (E)$

$E \rightarrow id$

Right Sentential Form	Handle	Reduction Production
$id1 + id2 * id3$	$id1$	$E \rightarrow id$
$E + id2 * id3$	$id2$	$E \rightarrow id$
$E + E * id3$	$id3$	$E \rightarrow id$
$E + E * E$	$E * E$	$E \rightarrow E * E$
$E + E$	$E + E$	$E \rightarrow E + E$
E		

(2.5)

Q.4. (d) LEX Tool

Ans: Lex is a computer program that generates lexical analyzers ("scanners" or "lexers").

Lex is commonly used with the yacc parser generator. Lex, originally written by Mike Lesk and Eric Schmidt and described in 1975, is the standard lexical analyzer generator on many Unix systems, and an equivalent tool is specified as part of the POSIX standard.

Lex reads an input stream specifying the lexical analyzer and outputs source code implementing the lexer in the C programming language

Open source

Though originally distributed as proprietary software, some versions of Lex are now open source. Open source versions of Lex, based on the original AT&T code are now distributed as open source systems such as OpenSolaris. One popular open

source version of Lex, called flex, or the "fast lexical analyzer", is not derived from proprietary coding.

Structure of a Lex file

The structure of a Lex file is intentionally similar to that of a yacc file; files are divided into three sections, separated by lines that contain only two percent signs, as follows

- The **definition** section defines macros and imports header files written in C. It is also possible to write any C code here, which will be copied verbatim into the generated source file.
- The **rules** section associates regular expression patterns with C statements. When the lexer sees text in the input matching a given pattern, it will execute the associated C code.
- The **C code** section contains C statements and functions that are copied to the generated source file. These statements presumably contain code called by the rules in the rules section. In large programs it is more convenient to place this code in a separate file linked in at compile time.

Example of a Lex file

The following is an example Lex file for the flex version of Lex. It recognizes strings of numbers (positive integers) in the input, and simply prints them out.

```

/** Definition section */
%{
/* C code to be copied verbatim */
#include <stdio.h>
%}
/* This tells flex to read only one input file */
%option noyywrap
%%
/** Rules section */
/* [0-9]+ matches a string of one or more digits */
[0-9]+ {
/* yytext is a string containing the matched text. */
printf("Saw an integer: %s\n", yytext);
}
| \n { /* Ignore all other characters. */ }
%%
/** C Code section */
int main(void)
{
/* Call the lexer, then quit. */
yylex();
return 0;
}

```

If this input is given to flex, it will be converted into a C file, lex.yy.c. This can be compiled into an executable which matches and outputs strings of integers. For example, given the input: